

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Fase 4: Iluminação, Normais e Texturas

Jorge Rafael Machado Fernandes
A104168

Diogo Teixeira Fernandes
A104260

Filipe Teixeira Viana
A104361

Data da Receção	
Responsável	
Avaliação	
Observações	

Fase 4: Iluminação, Normais e Texturas

maio, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Expansão do Formato <code>.3d</code>	2
2.2. Cálculo de Normais e Coordenadas de Textura para Primitivas	2
2.2.1. Plano	2
2.2.2. Caixa	2
2.2.3. Esfera	3
2.2.4. Cone	3
2.3. Superfícies de Bézier	3
2.3.1. Derivadas dos Polinómios de Bernstein	3
2.3.2. Cálculo de Normais via Produto Vetorial	4
2.3.3. Orientação dos Triângulos	4
3. Engine	5
3.1. Evolução das Estruturas de Dados	5
3.1.1. Vertex e <code>parse3DfileV2</code>	5
3.1.2. <code>ModelData</code> com 3 VBOs	5
3.1.3. Light	5
3.1.4. Material	5
3.2. Parsing do XML	6
3.2.1. Tag <code><lights></code>	6
3.2.2. Tag <code><texture></code>	6
3.2.3. Tag <code><color></code> com Sub-tags	6
3.2.4. Compatibilidade de Atributos	6
3.3. Iluminação	7
3.3.1. Modelo de Iluminação	7
3.3.2. <code>setupLights</code>	7
3.3.3. Aplicação de Materiais	7
3.3.4. Ambiente Global	7
3.4. Texturas	7
3.4.1. Carregamento com <code>DevIL</code>	7
3.4.2. Cache de Texturas	8
3.4.3. Aplicação na Renderização	8
3.5. Renderização Atualizada	8
3.5.1. Três VBOs por Modelo	8
3.5.2. Ordem de Bind dos Buffers	8
3.6. Exploração da Cena	9
4. Cena de Demonstração: Sistema Solar Iluminado	10
5. Utilitários e Estruturas de Dados	15
6. Conclusões	16

Lista de Figuras

Figura 1	Sistema solar iluminado com texturas planetárias, visto de dois ângulos distintos	11
Figura 2	Cubo iluminado por luz direcional	12
Figura 3	Cubo iluminado por luz direcional	12
Figura 4	Várias primitivas com material azul e luz pontual	13
Figura 5	Várias primitivas com material azul e luz pontual	13
Figura 6	Várias primitivas com material azul e spot	14
Figura 7	Várias primitivas com texturas distintas	14

1. Introdução

Este relatório foi elaborado no âmbito da Unidade Curricular de Computação Gráfica, no ano letivo de 2025/2026, e documenta o desenvolvimento da quarta e última fase do trabalho prático.

Nesta fase, estendemos o generator e o engine desenvolvidos nas fases anteriores, introduzindo três novas funcionalidades fundamentais: a geração de normais e coordenadas de textura por vértice em todas as primitivas, o suporte a fontes de luz (pontuais, direcionais e spotlight) com cálculo de iluminação por vértice, e a aplicação de materiais e texturas aos modelos da cena.

O formato dos ficheiros `.3d` foi estendido para armazenar oito valores por vértice (posição, normal e coordenadas de textura), e a pipeline de renderização foi reformulada para utilizar três Vertex Buffer Objects (VBOs) por modelo. A funcionalidade implementada foi validada através dos seis testes fornecidos pelo docente e da atualização da cena dinâmica do sistema solar com texturas planetárias.

2. Generator

O generator desenvolvido nas fases anteriores produzia ficheiros `.3d` com apenas a posição (x, y, z) de cada vértice. Nesta fase, foi estendido para também emitir a normal (n_x, n_y, n_z) e as coordenadas de textura (u, v) por vértice, totalizando oito valores por linha.

2.1. Expansão do Formato `.3d`

O formato dos ficheiros `.3d` foi alterado de três para oito colunas por vértice:

```
<numero_vertices>
x1 y1 z1 nx1 ny1 nz1 u1 v1
x2 y2 z2 nx2 ny2 nz2 u2 v2
...
```

Para suportar este novo formato, foi introduzida a estrutura `Vertex` no módulo partilhado, que agrega posição, normal e coordenadas de textura num único objeto. A função `saveToFile` foi sobrecarregada para aceitar vetores de `Vertex`, escrevendo as oito componentes em cada linha.

2.2. Cálculo de Normais e Coordenadas de Textura para Primitivas

Para cada primitiva foi necessário calcular a normal e as coordenadas de textura adequadas, com base na geometria específica da superfície.

2.2.1. Plano

Sendo o plano definido sobre o eixo XZ com a normal a apontar para cima, todos os vértices partilham a mesma normal:

$$\mathbf{n} = (0, 1, 0)$$

As coordenadas de textura mapeiam linearmente o plano $[-L/2, L/2] \times [-L/2, L/2]$ para o quadrado $[0, 1] \times [0, 1]$:

$$u = \frac{i}{\text{divisions}}, \quad v = \frac{j}{\text{divisions}}$$

onde i e j são os índices da subdivisão do plano.

2.2.2. Caixa

A caixa é composta por seis faces, cada uma com uma normal constante apontando para fora:

Face	Normal
Frente ($z = +L/2$)	$(0, 0, 1)$
Trás ($z = -L/2$)	$(0, 0, -1)$
Direita ($x = +L/2$)	$(1, 0, 0)$
Esquerda ($x = -L/2$)	$(-1, 0, 0)$
Cima ($y = +L/2$)	$(0, 1, 0)$
Baixo ($y = -L/2$)	$(0, -1, 0)$

Tabela 1: Normais constantes para cada face da caixa

As coordenadas de textura são mapeadas independentemente para cada face, com base na posição local de cada vértice dentro do quadrante da subdivisão.

2.2.3. Esfera

Para a esfera centrada na origem, a normal em qualquer ponto coincide com o vetor de posição normalizado. Para um ponto parametrizado por (θ, φ) com $\theta \in [0, 2\pi]$ (azimute) e $\varphi \in [-\pi/2, \pi/2]$ (elevação):

$$n(\theta, \varphi) = (\cos(\varphi) \sin(\theta), \sin(\varphi), \cos(\varphi) \cos(\theta))$$

As coordenadas de textura são definidas em função dos índices da grelha de slices e stacks:

$$u = \frac{\text{slice}}{\text{slices}}, \quad v = \frac{\text{stack}}{\text{stacks}}$$

Esta parametrização permite a aplicação direta de texturas equirretangulares, como as utilizadas para representar planetas.

2.2.4. Cone

Para a superfície lateral do cone com raio r e altura h , a normal num ponto à volta de um ângulo θ é dada por:

$$n(\text{lateral}) = \frac{h \sin(\theta), r, h \cos(\theta)}{\sqrt{h^2 + r^2}}$$

Esta normal é constante ao longo de cada slice (não depende da stack), uma vez que a inclinação da geratriz é a mesma a qualquer altura. A componente $y = r/\sqrt{h^2 + r^2}$ representa a inclinação para cima devida ao declive.

Para a base do cone, a normal é constante:

$$n(\text{base}) = (0, -1, 0)$$

As coordenadas de textura na superfície lateral são $u = \frac{\text{slice}}{\text{slices}}$ e $v = \frac{\text{stack}}{\text{stacks}}$. Na base, é aplicado um mapeamento circular centrado:

$$u = 0.5 + 0.5 \sin(\theta), \quad v = 0.5 + 0.5 \cos(\theta)$$

2.3. Superfícies de Bézier

A geração de normais para superfícies de Bézier exige o cálculo das derivadas parciais da superfície em ordem a u e v . A normal num ponto é então obtida pelo produto vetorial destas derivadas.

2.3.1. Derivadas dos Polinómios de Bernstein

Recordando que a superfície é definida por:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) \cdot B_j(v) \cdot P_{ij}$$

As derivadas parciais são calculadas analiticamente derivando a base de Bernstein em ordem ao parâmetro correspondente:

$$\frac{\partial S}{\partial u} = \sum_{i=0}^3 \sum_{j=0}^3 B'_i(u) \cdot B_j(v) \cdot P_{ij}$$

$$\frac{\partial S}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) \cdot B'_j(v) \cdot P_{ij}$$

As derivadas dos polinómios de Bernstein de grau 3 são:

$$B'_0(t) = -3(1-t)^2$$

$$B'_1(t) = 3(1-t)^2 - 6t(1-t)$$

$$B'_2(t) = 6t(1-t) - 3t^2$$

$$B'_3(t) = 3t^2$$

2.3.2. Cálculo de Normais via Produto Vetorial

A normal num ponto da superfície é obtida pelo produto vetorial das duas derivadas parciais, normalizado:

$$n(u, v) = \frac{\frac{\partial S}{\partial v} \times \frac{\partial S}{\partial u}}{\left\| \frac{\partial S}{\partial v} \times \frac{\partial S}{\partial u} \right\|}$$

A escolha da ordem do produto vetorial ($\partial v \times \partial u$ em vez do canónico $\partial u \times \partial v$) deve-se à convenção utilizada no ficheiro `teapot.patch`: os patches estão definidos de modo que o produto vetorial canónico produziria normais apontando para o interior da superfície. Invertendo a ordem, garantimos que a normal aponta para o exterior, o que é essencial para o correto cálculo da iluminação.

2.3.3. Orientação dos Triângulos

Para que a normal geométrica do triângulo (determinada pela ordem dos vértices na primitiva `GL_TRIANGLES`) seja consistente com a normal por vértice escrita no ficheiro `.3d`, os triângulos são emitidos com a seguinte ordem:

- **Triângulo 1:** P_1, P_3, P_2
- **Triângulo 2:** P_2, P_3, P_4

Esta orientação, combinada com o cálculo da normal como $\partial v \times \partial u$, garante que tanto a normal por vértice (utilizada na iluminação) como a normal geométrica (utilizada no back-face culling) apontam para o mesmo lado, evitando que faces visíveis sejam erradamente descartadas ou iluminadas pelo lado errado.

Para o teapot com 32 patches e tesselação 10, são gerados 19200 vértices, cada um com posição, normal e coordenadas de textura.

3. Engine

Esta fase exigiu uma reformulação significativa do engine: as estruturas de dados foram estendidas para acomodar normais, materiais e texturas; o parser de XML foi alargado para reconhecer luzes, materiais e referências a ficheiros de imagem; e a pipeline de renderização foi adaptada para utilizar três VBOs por modelo e aplicar iluminação OpenGL.

3.1. Evolução das Estruturas de Dados

3.1.1. Vertex e parse3DfileV2

Foi introduzida a estrutura `Vertex` no módulo partilhado, que agrega os oito valores por vértice. A função `parse3DfileV2` lê ficheiros `.3d` no novo formato, sendo tolerante a ficheiros antigos com apenas três valores por linha (que são lidos com normal (0, 1, 0) e UV (0, 0) por omissão), garantindo a compatibilidade com cenas das fases anteriores.

3.1.2. ModelData com 3 VBOs

A estrutura `ModelData` foi reformulada para suportar a nova pipeline:

- **vboPositions, vboNormals, vboTexCoords (GLuint):** identificadores dos três buffers OpenGL com a geometria, normais e coordenadas de textura
- **vertexCount (int):** número de vértices do modelo
- **material (Material):** propriedades do material associado ao modelo
- **textureFile (string):** ficheiro de textura associado (vazio se o modelo não tiver textura)
- **textureId (GLuint):** identificador da textura carregada para a GPU

A opção por três VBOs separados (em vez de um único VBO com dados interleaved) simplifica a interpretação dos dados pelos ponteiros `glVertexPointer`, `glNormalPointer` e `glTexCoordPointer`, e permite reutilizar os mesmos buffers em diferentes contextos.

3.1.3. Light

Para representar fontes de luz, foi introduzida a estrutura `Light` com um enumerador `LightType` que distingue os três tipos suportados:

- **LIGHT_POINT:** luz pontual posicional, definida por `posX`, `posY`, `posZ`
- **LIGHT_DIRECTIONAL:** luz direcional, definida por `dirX`, `dirY`, `dirZ`
- **LIGHT_SPOTLIGHT:** spotlight com posição, direção e ângulo de abertura (`cutoff`)

As luzes são armazenadas num vetor na estrutura `Configuration`, sendo aplicadas globalmente à cena no início de cada frame.

3.1.4. Material

A estrutura `Material` armazena as quatro componentes do modelo de Phong utilizado pelo OpenGL:

- **diffuse[4]:** cor difusa (resposta à luz dispersa)
- **ambient[4]:** cor ambiente (luz indireta)

- **specular[4]**: cor especular (reflexos brilhantes)
- **emissive[4]**: cor emissiva (luz emitida pelo próprio objeto)
- **shininess (float)**: expoente do brilho especular

Os valores por omissão correspondem aos indicados na especificação da fase (diffuse = 200/255, ambient = 50/255, restantes a zero).

3.2. Parsing do XML

3.2.1. Tag `<lights>`

A nova tag global `<lights>` permite definir uma ou mais fontes de luz na cena. Cada luz é representada por uma tag `<light>` com um atributo `type` que define o seu comportamento:

```
<lights>
  <light type="point" posX="0" posY="10" posZ="0" />
  <light type="directional" dirX="1" dirY="-1" dirZ="0" />
  <light type="spot" posX="0" posY="10" posZ="0"
        dirX="0" dirY="-1" dirZ="0" cutoff="30" />
</lights>
```

3.2.2. Tag `<texture>`

Cada modelo pode opcionalmente referenciar uma textura através da tag `<texture>` :

```
<model file="sphere.3d">
  <texture file="earth.jpg" />
</model>
```

O ficheiro de textura é procurado na pasta `textures/` e carregado para a GPU durante o parsing.

3.2.3. Tag `<color>` com Sub-tags

A tag `<color>` foi estendida para suportar sub-tags que definem as várias componentes do material:

```
<color>
  <diffuse R="200" G="200" B="200" />
  <ambient R="50" G="50" B="50" />
  <specular R="0" G="0" B="0" />
  <emissive R="0" G="0" B="0" />
  <shininess value="0" />
</color>
```

Os valores de RGB são esperados no intervalo $[0, 255]$ e são internamente normalizados para $[0, 1]$ antes de serem aplicados via `glMaterialfv`.

3.2.4. Compatibilidade de Atributos

Para garantir robustez ao parsing, foi implementada a função auxiliar `getAttrFlex` que aceita atributos tanto em camelCase (`posX`) como em minúsculas (`posx`), permitindo a leitura de cenas escritas por diferentes convenções sem necessidade de reescrita.

3.3. Iluminação

3.3.1. Modelo de Iluminação

Foi utilizado o modelo de iluminação fixo do OpenGL, que aplica a fórmula de Phong para cada vértice e interpola os resultados ao longo dos triângulos (Gouraud shading). A cor final de cada vértice é dada por:

$$I = I_{\text{emissive}} + I_{\text{ambient}} + \sum_l (I_{\text{diffuse}}^l + I_{\text{specular}}^l)$$

onde a componente ambiente combina a cor ambiente do material com a luz ambiente global da cena, e as componentes difusa e especular são calculadas por luz l em função do vetor normal e da direção da luz.

3.3.2. `setupLights`

A função `setupLights`, invocada no início de cada frame após a definição da câmara, configura todas as luzes da cena. Cada luz é ativada (`glEnable(GL_LIGHTn)`) e os seus parâmetros são definidos com `glLightfv` :

- **Luz pontual:** `GL_POSITION` com $w = 1$ (posicional) e atenuação desativada para garantir iluminação uniforme em toda a cena
- **Luz direcional:** `GL_POSITION` com $w = 0$, interpretado pelo OpenGL como direção a partir da qual a luz incide
- **Spotlight:** combinação de posição ($w = 1$), `GL_SPOT_DIRECTION` e `GL_SPOT_CUTOFF`

3.3.3. Aplicação de Materiais

Antes do desenho de cada modelo, as quatro componentes do material são aplicadas via `glMaterialfv` com `GL_FRONT_AND_BACK`, garantindo que tanto a face frontal como a traseira recebem o mesmo material. O expoente de brilho é aplicado com `glMaterialf(GL_SHININESS, ...)`.

3.3.4. Ambiente Global

A luz ambiente global é definida em `main.cpp` com o valor (1.0, 1.0, 1.0, 1.0), conforme recomendado pelo docente:

```
float globalAmb[4] = {1.0f, 1.0f, 1.0f, 1.0f};
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, globalAmb);
```

Este valor permite que a cor ambiente definida no material de cada modelo seja reproduzida na sua totalidade, sem ser atenuada pelo modelo de iluminação global.

Adicionalmente, é ativado `GL_RESCALE_NORMAL`, que renormaliza automaticamente as normais após a aplicação de transformações de escala uniforme, garantindo que o cálculo da iluminação se mantém correto mesmo em grupos com `<scale>`.

3.4. Texturas

3.4.1. Carregamento com DevIL

Para o carregamento de imagens em diferentes formatos (JPG, PNG, etc.) foi integrada a biblioteca **DevIL** (Developer's Image Library). A biblioteca é inicializada uma vez no arranque da aplicação com `ilInit()` e utilizada para ler cada ficheiro de imagem:

```

ilGenImages(1, &ilImg);
ilBindImage(ilImg);
ilLoadImage(filepath);
ilConvertImage(IL_RGBA, IL_UNSIGNED_BYTE);

```

Os dados são depois transferidos para uma textura OpenGL com `glTexImage2D`. Os parâmetros de filtragem são definidos como `GL_LINEAR` para minificação e magnificação, garantindo uma renderização nítida sem o efeito de desfoque introduzido por mipmaps em texturas de baixa frequência (como a relva ou o padrão do teapot).

3.4.2. Cache de Texturas

Para evitar o carregamento redundante de imagens referenciadas por múltiplos modelos, foi implementado um cache simples baseado num `std::map<std::string, GLuint>` que associa o caminho do ficheiro ao identificador da textura na GPU. Se uma textura já tiver sido carregada anteriormente, o cache devolve diretamente o `textureId` existente.

3.4.3. Aplicação na Renderização

Para cada modelo com textura, durante o desenho:

```

glEnable(GL_TEXTURE_2D);
glBindTexture(GL_TEXTURE_2D, m.textureId);
glBindBuffer(GL_ARRAY_BUFFER, m.vboTexCoords);
glTexCoordPointer(2, GL_FLOAT, 0, 0);
glEnableClientState(GL_TEXTURE_COORD_ARRAY);

```

Modelos sem textura desativam `GL_TEXTURE_2D` e o `GL_TEXTURE_COORD_ARRAY` antes de serem desenhados, sendo coloridos apenas pelo material.

3.5. Renderização Atualizada

3.5.1. Três VBOs por Modelo

Durante o parsing, cada modelo gera três buffers OpenGL distintos: um para as posições (3 floats por vértice), outro para as normais (3 floats) e outro para as coordenadas de textura (2 floats). Esta separação simplifica a sua atribuição via `glVertexPointer`, `glNormalPointer` e `glTexCoordPointer`.

3.5.2. Ordem de Bind dos Buffers

O desenho de cada modelo segue uma sequência fixa para garantir consistência:

- Aplicação do material (`glMaterialfv` para cada componente)
- Bind da textura (se existir) e do VBO de coordenadas de textura
- Bind do VBO de normais e definição do ponteiro de normais
- Bind do VBO de posições e definição do ponteiro de vértices
- `glDrawArrays(GL_TRIANGLES, 0, vertexCount)`

3.6. Exploração da Cena

Foram adicionados controlos interativos a complementar os das fases anteriores:

- **Tecla ** : ativa/desativa a visualização das normais por vértice, desenhadas como segmentos vermelhos a partir de cada vértice
- **FPS counter**: exibido no título da janela, atualizado a cada segundo
- **Limite na elevação da câmara**: o ângulo de elevação é restringido ao intervalo $[-1.5, 1.5]$ rad, evitando o flip da câmara nos pólos

4. Cena de Demonstração: Sistema Solar Iluminado

A cena de demonstração desta fase evolui o sistema solar dinâmico das fases anteriores, agora com iluminação realista e texturas planetárias.

A iluminação é fornecida por uma única luz pontual posicionada na origem da cena, coincidente com o Sol. A luz pontual tem atenuação desativada, permitindo que todos os planetas, mesmo os mais distantes, sejam iluminados uniformemente. O Sol em si possui uma componente emissiva forte, o que o faz brilhar com a sua própria cor, independentemente da luz da cena.

Cada planeta utiliza uma textura realista obtida do website *Solar System Scope*, mapeada à esfera através das coordenadas (u, v) esféricas calculadas pelo generator. Os materiais foram afinados individualmente:

- **Sol:** emissivo forte, sem reflexão especular
- **Terra:** especular moderado para simular o reflexo dos oceanos
- **Restantes planetas:** difuso predominante com ambiente moderado, sem especular

O cometa (teapot de Bézier), introduzido na Fase 3, foi mantido com a sua trajetória de Catmull-Rom, mas agora possui um material com forte componente especular ($\text{shininess} = 100$), evidenciando o seu brilho à medida que percorre a órbita.

A tabela seguinte resume a estrutura hierárquica atualizada do sistema solar, indicando o modelo, as transformações, a textura aplicada e os subgrupos associados a cada corpo.

Corpo	Modelo	Textura	Subgrupos
Sol	sphere (r=1, 20×20)	sun.jpg	—
Mercúrio	sphere (r=1, 20×20)	mercury.jpg	—
Vénus	sphere (r=1, 20×20)	venus.jpg	—
Terra	sphere (r=1, 20×20)	earth.jpg	Lua
Marte	sphere (r=1, 20×20)	mars.jpg	Fobos, Deimos
Júpiter	sphere (r=1, 20×20)	jupiter.jpg	Io, Europa, Ganímedes, Calisto
Saturno	sphere (r=1, 20×20)	saturn.jpg	Anel, Titã
Urano	sphere (r=1, 20×20)	uranus.jpg	—
Neptuno	sphere (r=1, 20×20)	neptune.jpg	—
Cometa	teapot (bezier, tess=10)	—	—

Tabela 2: Estrutura do sistema solar com texturas aplicadas

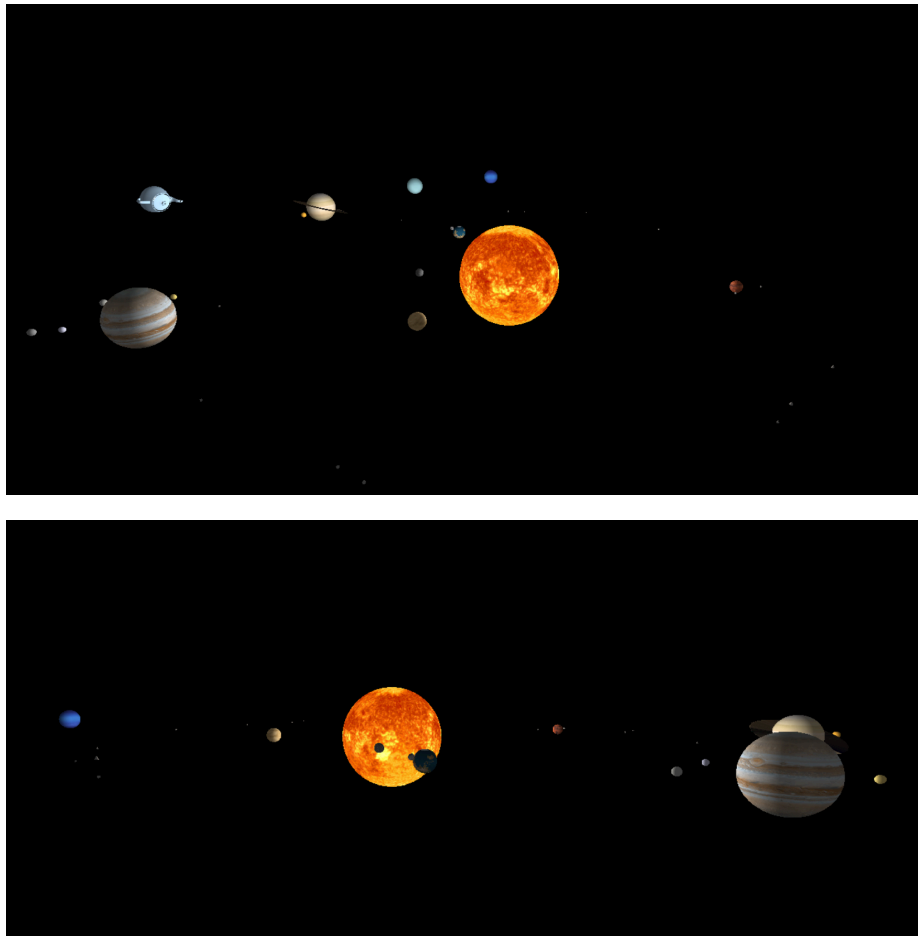


Figura 1: Sistema solar iluminado com texturas planetárias, visto de dois ângulos distintos

Os testes foram igualmente validados:

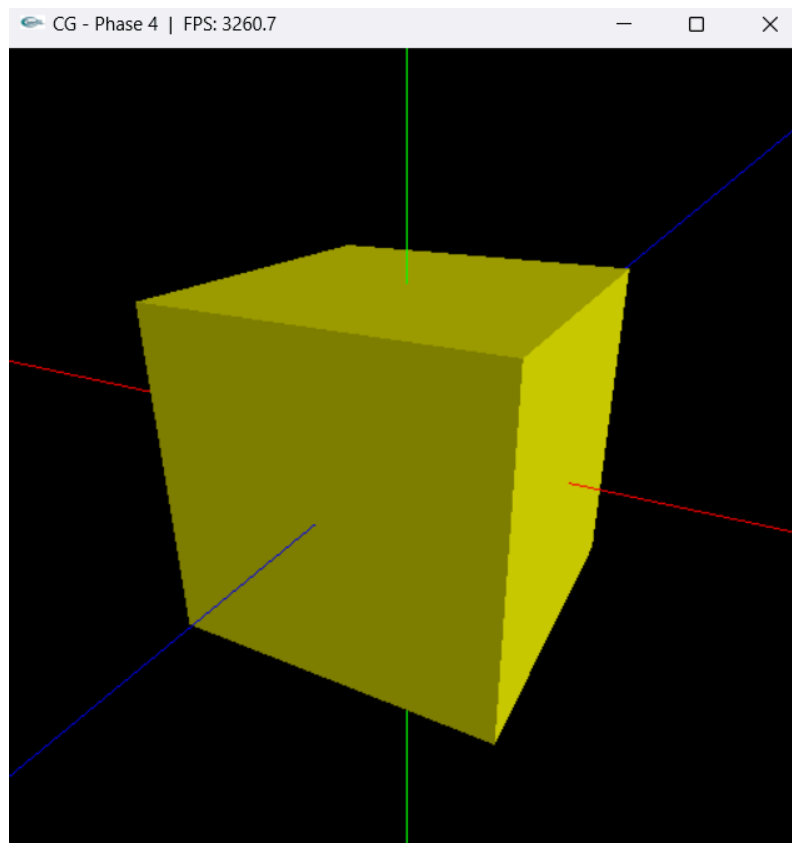


Figura 2: Cubo iluminado por luz direcional

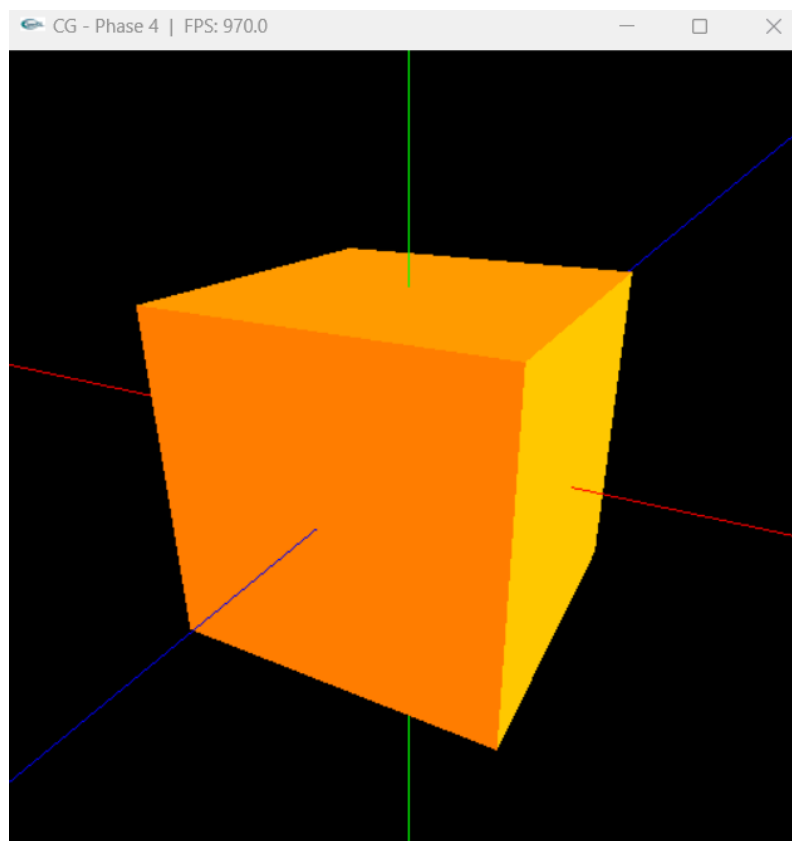


Figura 3: Cubo iluminado por luz direcional

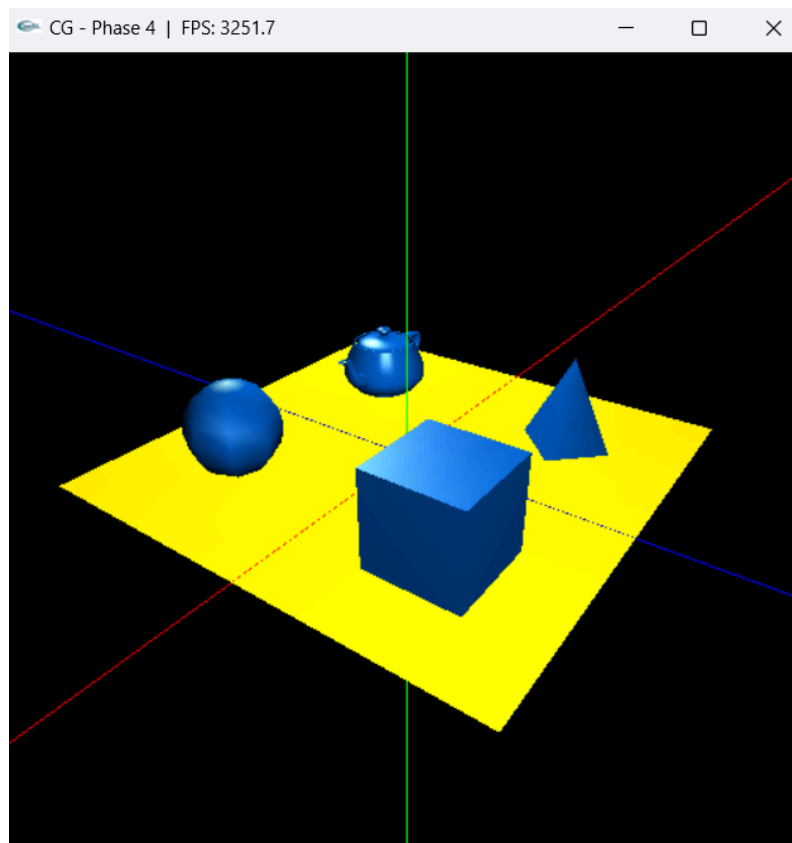


Figura 4: Várias primitivas com material azul e luz pontual

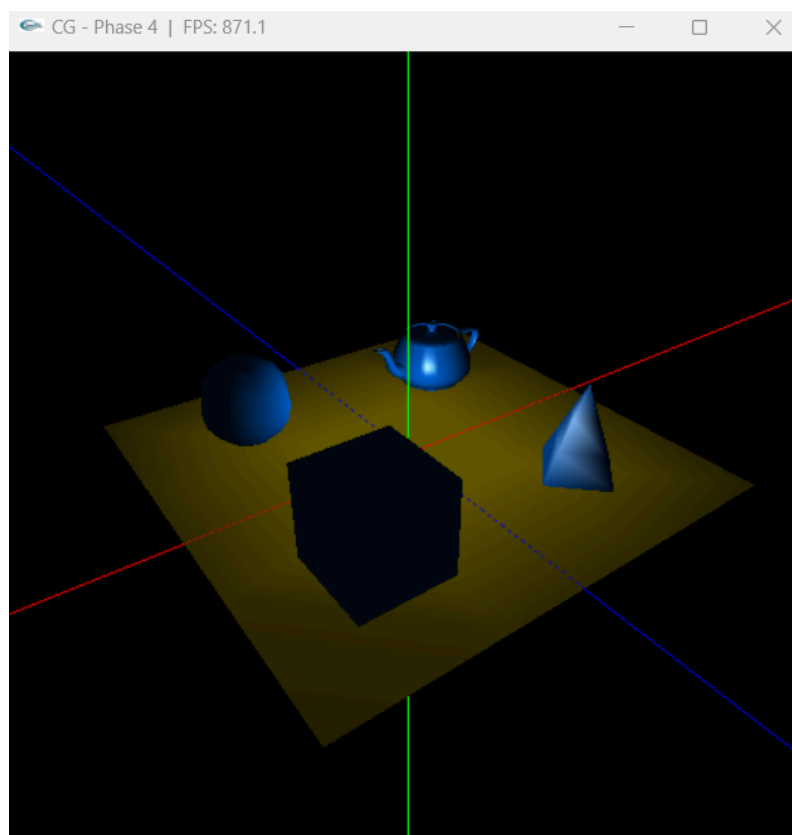


Figura 5: Várias primitivas com material azul e luz pontual

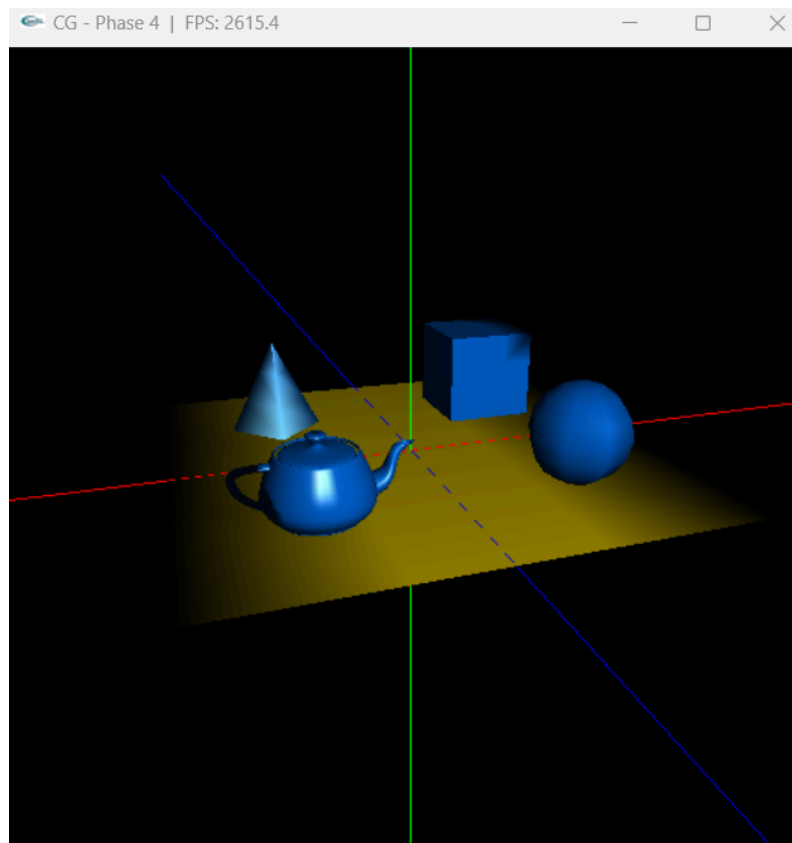


Figura 6: Várias primitivas com material azul e spot

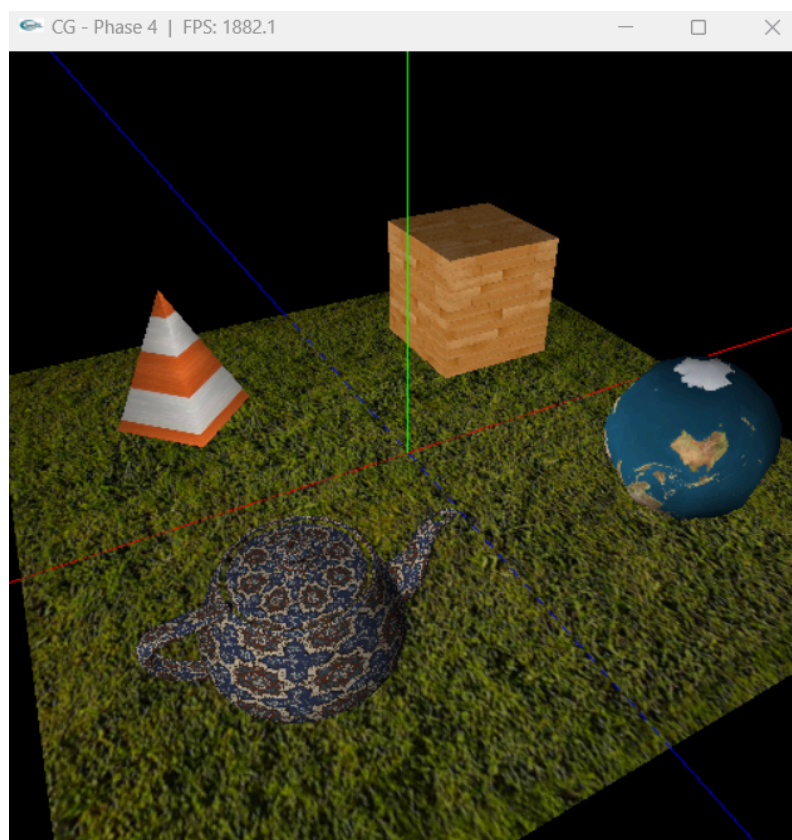


Figura 7: Várias primitivas com texturas distintas

5. Utilitários e Estruturas de Dados

Ao nível das bibliotecas externas, foi adicionada a **DevIL** (Developer's Image Library) para o carregamento de imagens em formatos JPG e PNG. A integração foi feita à semelhança da GLEW na Fase 3: os ficheiros da biblioteca foram colocados na pasta `toolkits/devil/` e o `CMakeLists.txt` foi alterado para incluir os headers, ligar a biblioteca estática e copiar a DLL automaticamente após o build.

Ao nível das estruturas partilhadas, manteve-se a classe `Point` para coordenadas 3D e introduziu-se a estrutura `Vertex` para o novo formato de oito valores por vértice. As funções `saveToFile` e `parse3DfileV2` lidam respetivamente com a escrita e leitura do novo formato, com retro-compatibilidade para ficheiros antigos.

No engine, as estruturas `Light` e `Material` foram introduzidas para representar fontes de luz e materiais. A classe `Configuration` foi estendida com um vetor de `Light` que armazena todas as luzes da cena, e a estrutura `ModelData` foi reformulada com três identificadores de VBO (posições, normais e coordenadas de textura), além do identificador da textura e do material associado.

6. Conclusões

Nesta fase, completámos o motor gráfico com iluminação, normais por vértice e texturas, finalizando o conjunto de funcionalidades previstas na especificação.

A alteração mais transversal foi a extensão do formato `.3d` de três para oito valores por vértice. Apesar de parecer uma mudança simples, acabou por obrigar a alterar várias partes do projeto: no generator, passámos a calcular normais e coordenadas de textura para cada primitiva, sendo o caso das superfícies de Bézier o mais trabalhoso, por exigir o cálculo das derivadas parciais; no engine, foi necessário passar de um VBO para três, adicionar o parsing de materiais e luzes, e integrar a biblioteca DevIL para carregar imagens.

Com as texturas e a iluminação aplicadas, a cena do sistema solar ficou bastante mais expressiva. Os planetas passaram a usar texturas reais e o Sol funciona agora como a única fonte de luz da cena. Os testes do docente também foram úteis, porque permitiram validar isoladamente cada tipo de luz e cada componente dos materiais.

Durante a implementação surgiram alguns problemas que vale a pena registar. O `GL_RESCALE_NORMAL` teve de ficar ativo, caso contrário as escalas aplicadas aos grupos distorciam as normais. O valor de ambiente global também teve de ser definido como `1.0`, em vez do valor por defeito, para que as cores ambiente dos materiais fossem reproduzidas corretamente. No caso do Bézier, descobrimos ainda que o produto vetorial das derivadas tinha de ser feito na ordem inversa, isto é, $(\partial v / \partial u \times \partial v / \partial v)$. Com a convenção usada no `teapot.patch`, a ordem contrária fazia com que a normal canónica apontasse para dentro da superfície, o que inicialmente deixava o bule escurecido e mal iluminado.

No final desta fase, o motor suporta as principais funcionalidades exigidas: hierarquia de grupos, transformações estáticas e dinâmicas, primitivas e superfícies de Bézier, curvas de Catmull-Rom, VBOs, iluminação, materiais e texturas.